

SQL Technical Interview Practice Exam

10 Real-World Interview Problems & Target Execution Outputs

Target Database: `test-company.db`

September 6, 2026

Practice Instructions & Database Schema

Use this practice document to write and test your SQL queries independently. Compare your query results against the verified benchmark outputs provided for each question.

- **employees** (2,500 records): `id, name, email, salary, status, hire_date, department_id, pay_type_id, salary_grade_id`
- **departments** (10 records): `id, department_name`
- **pay_types** (2 records): `id, description` (*Hourly, Salary*)
- **salary_grades** (5 records): `id, description, long_text`
- **customers** (100 records): `id, company_name, representative_id`
- **transactions** (5,000 records): `id, customer_id, service_type, transaction_type, sales_type, net_amount`

Contents

1 Practice Questions & Benchmark Outputs	2
Question 1: Basic Aggregation Filtering: Active Employee Count and Average Salary . .	2
Question 2: Aggregated Filtering: High-Headcount Departments via HAVING	2
Question 3: Subqueries & CTEs: Employees Earning Above Department Average	2
Question 4: Window Functions: Top 2 Highest-Paid Employees per Department	3
Question 5: Conditional Aggregation: Pivoting Sales Channels by Service Type	3
Question 6: Multi-Table Relational Joins: Top Sales Representatives by Revenue	4
Question 7: LEFT JOIN & Status Breakdown: Salary Grade Distributions	4
Question 8: Date Handling & Cohort Retention Rate Analysis	5
Question 9: Cumulative Window Functions: Running Balance of Transactions	5
Question 10: Customer Segmentation & Quartile Analysis via NTILE()	6

1 Practice Questions & Benchmark Outputs

Question 1: Basic Aggregation & Filtering: Active Employee Count and Average Salary

Tested SQL Concepts: Basic Aggregations (COUNT, AVG, ROUND), WHERE filter, JOIN, GROUP BY

Problem Statement:

Write an SQL query to retrieve the total count of currently **Active** employees and their average salary (rounded to 2 decimal places) grouped by their pay type (e.g., Hourly vs Salary). Sort the result by average salary in descending order.

Hints & Considerations:

Ensure you filter active records prior to grouping and join with the `pay_types` table to retrieve descriptive names.

Expected Output (Target Reference):

pay_type	active_employee_count	avg_salary
Salary	619	115371.58
Hourly	624	112674.99

Question 2: Aggregated Filtering: High-Headcount Departments via HAVING

Tested SQL Concepts: GROUP BY, HAVING clause vs. WHERE clause, Multi-column Aggregations (MIN, MAX)

Problem Statement:

Find all company departments that have **at least 125 active employees**. For each qualifying department, output the department name, the number of active employees, the lowest salary, and the highest salary in that department. Sort descending by employee count.

Hints & Considerations:

Remember the order of operations: filter for active status before grouping, then use HAVING to filter groups by headcount.

Expected Output (Target Reference):

department_name	active_employee_count	min_salary	max_salary
Marketing	136	31223	199411
Finance	132	31476	198971
Executive	131	30189	198537
Communications	131	30002	199563
Customer Service	125	30002	199297

Question 3: Subqueries & CTEs: Employees Earning Above Department Average

Tested SQL Concepts: Common Table Expressions (WITH), Two-stage Aggregations, Conditional Join Matching

Problem Statement:

Identify active employees whose salary exceeds the average salary of their specific department. Return the employee name, department name, employee salary, and the department average salary (rounded to 2 decimals). Order by employee salary descending and limit output to the top 5.

Hints & Considerations:

Consider precomputing department average salaries using a CTE (`WITH` clause) before joining it back to individual employee records.

Expected Output (Target Reference):

employee_name	department_name	salary	dept_avg_salary
Sofia Roberts	Information Technology	199690	113535.64
Evelyn Lopez	Information Technology	199600	113535.64
Ava Adams	Communications	199563	108471.4
Sofia Jackson	Marketing	199411	112819.71
Nora Baker	Customer Service	199297	115006.68

Question 4: Window Functions: Top 2 Highest-Paid Employees per Department

Tested SQL Concepts: Window Functions (`DENSE_RANK()`), `PARTITION BY`, `ORDER BY` inside window, CTE filtering

Problem Statement:

For each department, find the top 2 highest-paid active employees using the `DENSE_RANK()` window function. Return the department name, employee name, salary, and their departmental rank. Display the first 10 rows sorted alphabetically by department and rank.

Hints & Considerations:

Window functions cannot be evaluated directly in a `WHERE` clause; wrap the ranked query in a CTE or subquery.

Expected Output (Target Reference):

department_name	employee_name	salary	salary_rank
Communications	Ava Adams	199563	1
Communications	Oliver Lee	199077	2
Customer Service	Nora Baker	199297	1
Customer Service	Leo Adams	197888	2
Executive	Ava Lee	198537	1
Executive	John Jackson	197811	2
Finance	Levi Scott	198971	1
Finance	Luca Wilson	197295	2
Information Technology	Sofia Roberts	199690	1
Information Technology	Evelyn Lopez	199600	2

Question 5: Conditional Aggregation: Pivoting Sales Channels by Service Type

Tested SQL Concepts: Conditional Aggregation (`CASE WHEN` inside `SUM`), Cross-tabulation / Pivoting

Problem Statement:

For each `service_type` in transactions, calculate the total revenue generated through 'Online' sales, 'Phone' sales, and the combined overall revenue. Round all monetary values to 2 decimal places and sort by total revenue descending.

Hints & Considerations:

Use `CASE WHEN sales_type = 'Online' THEN net_amount ELSE 0 END` inside aggregate `SUM()` functions.

Expected Output (Target Reference):

service_type	online_revenue	phone_revenue	total_revenue
Service	2711019.43	2403741.7	5114761.13
Maintenance	2348240.26	2672761.31	5021001.57
Consulting	2461679.51	2474814.49	4936494.0
One-time	2270628.11	2589030.31	4859658.42
Contract	2302487.79	2444295.47	4746783.26

Question 6: Multi-Table Relational Joins: Top Sales Representatives by Revenue

Tested SQL Concepts: 4-Table Inner Joins (`employees`, `departments`, `customers`, `transactions`), `COUNT(DISTINCT)`

Problem Statement:

Find the top 5 employee account representatives whose assigned customers generated the highest total transaction revenue. Display representative name, department name, number of distinct customers managed, and total revenue generated.

Hints & Considerations:

Connect `employees` to `customers` and `transactions`; use `COUNT(DISTINCT c.id)` to count unique clients.

Expected Output (Target Reference):

representative_name	department_name	managed_customers_count	total_revenue_generated
Henry Hill	Customer Service	1	288351.34
Harper Davis	Risk Management	1	285654.47
Violet Taylor	Purchasing	1	285309.34
Charlotte Martin	Payroll	1	284945.14
Noah Williams	Purchasing	1	284196.04

Question 7: LEFT JOIN & Status Breakdown: Salary Grade Distributions

Tested SQL Concepts: `LEFT JOIN`, Handling Non-Matching Outer Data, Pivot Aggregations

Problem Statement:

List all salary grades (ID, tier description, and full title) along with the count of active employees, terminated employees, and total employees assigned to each grade. Ensure all salary grades are preserved in the result even if an employee count is 0.

Hints & Considerations:

Use LEFT JOIN from salary_grades to employees and conditional counting (COUNT(CASE WHEN ...)).

Expected Output (Target Reference):

grade_id	grade_tier	grade_title	active_count	terminated_count	total_employees
1	Tier 1	Worker	375	350	725
2	Tier 2	Lower Management	138	154	292
3	Tier 3	Middle Management	162	156	318
4	Tier 4	Upper Management	158	168	326
5	Tier 5	Senior Management	410	429	839

Question 8: Date Handling & Cohort Retention Rate Analysis

Tested SQL Concepts: Date pattern parsing, Categorical Cohorting, Percentage Metrics, Integer-to-Float Division

Problem Statement:

Group employees into hiring cohorts by decade (1980s, 1990s, 2000s, 2010s, 2020s). Calculate total hired, count of currently active staff, and the active retention rate percentage (Active / Total * 100, rounded to 2 decimal places).

Hints & Considerations:

Categorize cohorts using CASE WHEN hire_date LIKE ... and multiply by 100.0 to prevent integer division truncation.

Expected Output (Target Reference):

hire_decade	total_hired	active_employees	retention_rate_pct
1980s	87	46	52.87
1990s	248	114	45.97
2000s	435	218	50.11
2010s	809	402	49.69
2020s	920	463	50.33

Question 9: Cumulative Window Functions: Running Balance of Transactions

Tested SQL Concepts: Cumulative Window Framing (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), Window SUM()

Problem Statement:

For Customer ID = 1 (NVIDIA), list their first 6 transactions ordered by transaction ID. Display transaction ID, service type, transaction type, sales type, transaction net amount, and the running cumulative total amount spent up to that transaction.

Hints & Considerations:

Use SUM(net_amount) OVER (ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) to compute cumulative totals.

Expected Output (Target Reference):

transaction_id	service_type	transaction_type	sales_type	net_amount	running_total_amount
37	One-time	Buy	Phone	9182.39	9182.39
88	Consulting	Transfer	Online	1800.85	10983.24
182	Contract	Sale	Online	7186.42	18169.66
231	One-time	Transfer	Phone	6700.46	24870.12
285	One-time	Buy	Phone	9429.71	34299.83
305	Service	Transfer	Phone	4283.43	38583.26

Question 10: Customer Segmentation & Quartile Analysis via NTILE()

Tested SQL Concepts: Statistical Ranking (NTILE(4)), Customer Lifetime Value Aggregation, CTE

Problem Statement:

Divide all 100 enterprise customers into 4 spending quartiles (where Quartile 1 contains the top 25% highest spenders) based on their total transaction volume. Display the top 5 highest-spending companies in Quartile 1.

Hints & Considerations:

Compute total spend per customer in a CTE, apply NTILE(4) OVER (ORDER BY SUM(t.net_amount) DESC), then filter for quartile 1.

Expected Output (Target Reference):

customer_id	company_name	total_spend	spending_quartile
68	AppLovin	288351.34	1
4	Microsoft	285654.47	1
51	Verizon	285309.34	1
79	Honeywell	284945.14	1
30	Coca-Cola	284196.04	1